

Class 11: Advanced state management with Redux

2110507 Software Development Practice II

Machigar Ongtang, Ph.D.

Redux



- State Management บริหารจัดการข้อมูลระหว่าง session การใช้งานของ user
  - กลไกสำคัญในการบริหารจัดการ user data และจดจำ interaction ของ user ที่กระทำไว้ต่อ web application เพื่อให้ application สามารถเข้าถึงและประมวลผล data เหล่านั้นต่อได้
    - สร้าง Shopping cart สำหรับ e-commerce website
    - จดจำ user session หลังการ authentication

- State ใน Next.js application แบ่งได้เป็น
  - Global State เก็บ information ที่ share ระหว่าง component ทั้งหมดใน application เช่น authentication status ของ user
  - Component State เก็บ data สำหรับแต่ละ component

- Redux เป็นหนึ่งใน state management solution สำหรับ Java Script Framework ต่าง ๆ
  - State Management Library สำหรับ front-end JavaScript application
  - แยก code สำหรับจัดการและจัดเก็บ data ออกจาก code ที่จัดการ event และ effect บน component

Redux Concepts



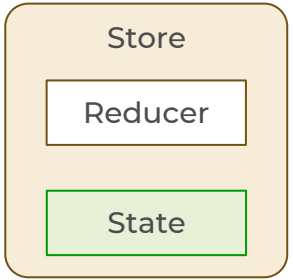
- Redux เป็น pattern และ library สำหรับ manage และ update application state โดยใช้ event ต่าง ๆ ที่เรียกว่า action
  - Redux ทำหน้าที่เป็น store กลางสำหรับจัดเก็บ state ที่ share ใช้งานทั่วทั้ง application โดยจัดการ Global State
  - Redux มาพร้อม rule ต่าง ๆ เพื่อจัดการให้การ update state กระทำในลักษณะที่คาดการณ์ได้ (predictable)
    - Redux มาพร้อม pattern และ tool ต่าง ๆ ที่ช่วยให้ developer เข้าใจการ update ของ state ใน application ว่าเกิดขึ้นเมื่อไร (when) ที่ไหน (where) เพราะเหตุใด (why) และอย่างไร (how) และเข้าใจพฤติกรรมของ application logic เมื่อเกิดการ update เหล่านั้น
    - เอื้อต่อการ coding ที่คาดการณ์ได้ (predictable) และทดสอบได้ ให้พฤติกรรมที่เป็นไปตามความคาดหวัง
  - อย่างไรก็ตาม การใช้งาน Redux เพิ่มความซับซ้อนให้ application จึงเหมาะกับ application ที่
    - มี application state จำนวนมาก ใช้งานทั่ว application
    - มีการ update application state สม่ำเสมอ และอาจใช้ logic ที่ซับซ้อน
    - มี codebase ขนาดกลางหรือใหญ่ ทำงานกับทีมหลายคน



Redux Concepts – *How it works?*

Store

- Store เป็น center ของ Redux application โดยเป็น container ที่จัดเก็บ global state ของ application โดยเป็น JavaScript object ที่มี function และคุณสมบัติพิเศษกว่า Global JavaScript Object ทั่วไป
  - Developer จะไม่ modify หรือเปลี่ยน state ที่เก็บอยู่ใน Redux store โดยตรง
  - การ modify หรือ update state กระทำผ่าน **action** object ที่ระบุว่าสิ่งที่เกิดขึ้นใน application โดย **dispatch** action ไปยัง store เพื่อให้ทราบถึงสิ่งที่เกิดขึ้น
  - เมื่อ action ถูก dispatch ไปยัง store จะทำให้ store ทำการ run root **reducer** function เพื่อประมวลผล state ให้จาก state เดิมและจาก action
  - สุดท้าย store ทำการแจ้ง notify ไปยัง **subscriber** ว่า state ได้รับการ update เพื่อให้ UI ได้รับการ update ด้วย data ใหม่



Redux Concepts – *How it works?*



- **Action** เป็น plain JavaScript object เทียบเคียงกับ event ที่ระบุสิ่งที่เกิดขึ้นใน application.
  - Action บรรจุ type field เป็น string กำหนดชื่อที่อธิบาย action ที่เกิดขึ้น
    - นิยมระบุในรูปแบบ `domain/eventName` โดย `domain` ระบุ feature หรือ category ของ action และ `eventName` ระบุสิ่งที่เกิดขึ้น เช่น `todos/todoAdded`
  - Field ที่ระบุ information เพิ่มเติมเกี่ยวกับสิ่งที่เกิดขึ้น มักตั้งชื่อ field ว่า `payload`

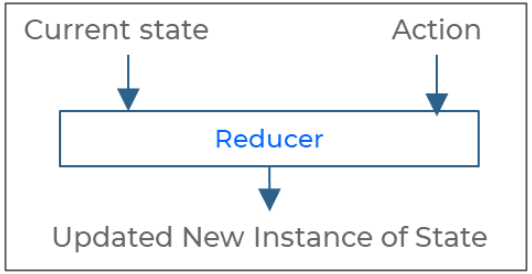
```
const addTodoAction = {
  type: 'todos/todoAdded',
  payload: 'Buy milk'
}
```

- **Action Creator** เป็น function ที่สร้าง และ return action object
  - ใช้งาน Action Creator เพื่อหลีกเลี่ยงการเขียน action object ทุกครั้งที่ต้องการสร้าง action

```
const todoAdded = text => {
  return {
    type: 'todos/todoAdded',
    payload: text
  }
}
```

Redux Concepts – *How it works?*

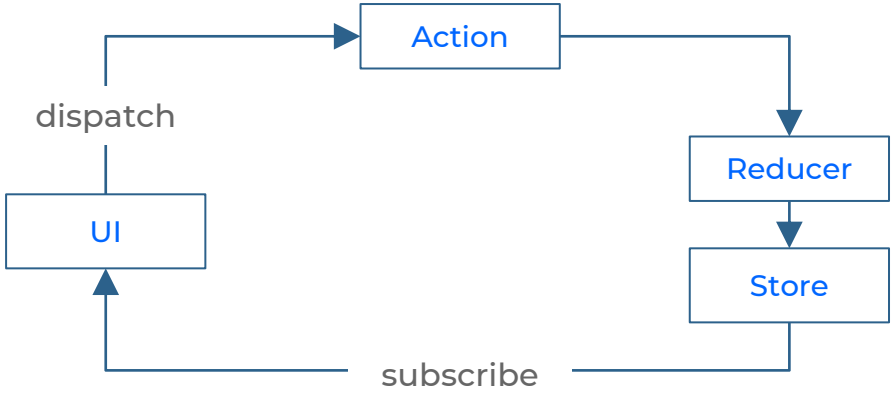
- Reducer เป็น function ที่รับ `state` ปัจจุบัน และ `action` object เพื่อตัดสินรูปแบบการ update state ถ้าจำเป็น แล้ว return ค่า state ใหม่
  - `(state, action) => newState`
  - Reducer เทียบได้กับ event listener ที่ handle event โดยพิจารณาจาก action (event) `type` ที่ได้รับ
  - Reducer จะต้องดำเนินการตาม rule
    - ควร calculate เพื่อให้ได้ value ของ state ใหม่ เท่านั้น โดยพิจารณาจาก state และ action arguments
    - ไม่ modify state เดิมที่มี แต่ทำ immutable update โดยการ copy ค่า state เดิม แล้วทำการเปลี่ยนแปลงต่อค่าที่ copy ไป
    - ไม่ทำ asynchronous logic, ไม่ calculate ค่า random value, ไม่สร้าง side effects ใด
  - Logic ใน reducer function มักประกอบด้วยขั้นตอนต่อไปนี้
    - ตรวจสอบว่า reducer สนใจ หรือเกี่ยวข้องกับ action ที่รับมาหรือไม่ ถ้าสนใจ จะทำ copy ของ state ทำการ update ตัว copy ด้วย value ใหม่ แล้ว return ค่าออกมา
    - ถ้าไม่สนใจ จะ return ค่า state เดิมออกมา



Redux Concepts – *How it works?*

- **Dispatch** เป็น method ชื่อ dispatch ใน Store
  - เมื่อเมธอด `dispatch` ของ Store ถูกเรียก และมีการส่ง action object มาให้ จะทำให้ store ทำการ run reducer function และบันทึก value ของ state ใหม่ไว้ใน store
  - การ dispatch action เทียบได้กับการ trigger event

- **Selector** เป็น function ต่าง ๆ ที่ใช้ extract ชิ้นของ information จาก value ของ state ใน store
  - เป็น logic ที่ทำให้ส่วนต่าง ๆ ของ application เรียกใช้ได้สะดวก หลีกเลี่ยงการเขียน logic เดิมซ้ำ ๆ



Redux Concepts – *How it works?*



- Redux ทำงานภายใต้หลักการและแนวคิดต่อไปนี้

Single Source of Truth

- Global state ของ application จัดเก็บเป็น object ใน single store
- Data แต่ละชิ้นจัดเก็บที่ location เดียว ไม่มีการ duplicate ไปหลายที่
- ง่ายต่อการ debug และ inspect ค่า state เมื่อมีการเปลี่ยนแปลง และมี logic กลางในการ interact กับส่วนต่าง ๆ ของ application

State เป็นแบบ Read-Only

- การเปลี่ยนแปลง state ทำโดยการ action หลีกเลี่ยงไม่ให้ UI ทำการ overwrite data โดยบังเอิญ
- ง่ายต่อการ trace หาสาเหตุของการ update ค่า state
  - Action เป็น plain JS objects สามารถ log, serialized, stored และใช้ในการ debug หรือ test ภายหลังได้

เปลี่ยนแปลงผ่าน Pure Reducer Function

- Reducer functions ระบุรูปแบบการ update สำหรับ state tree โดยเป็น pure function ที่รับค่า state ก่อนหน้า และ action แล้ว return ค่า state ต่อไป
- สามารถแยก reducer ออกเป็น function ย่อย เพื่อสร้างเป็นฟังก์ชั่นที่ reusable สำหรับ common task

Redux Concepts – *How it works?*

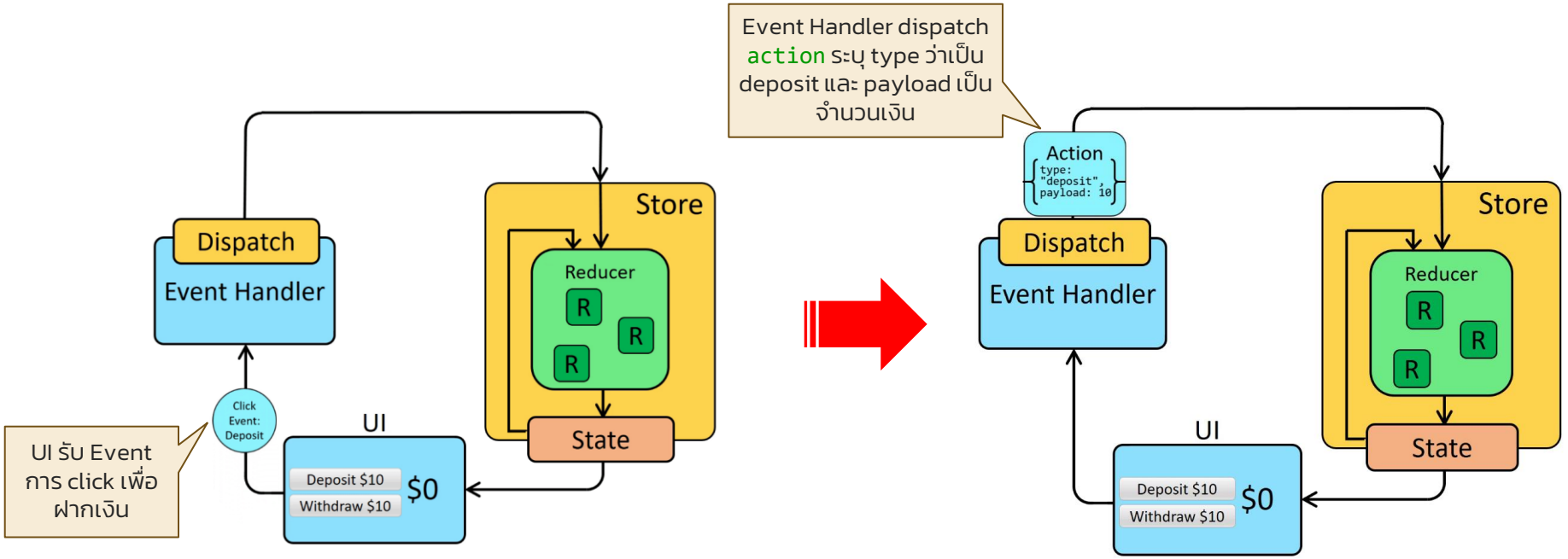


Redux มีการทำงานตาม Data Flow ต่อไปนี้

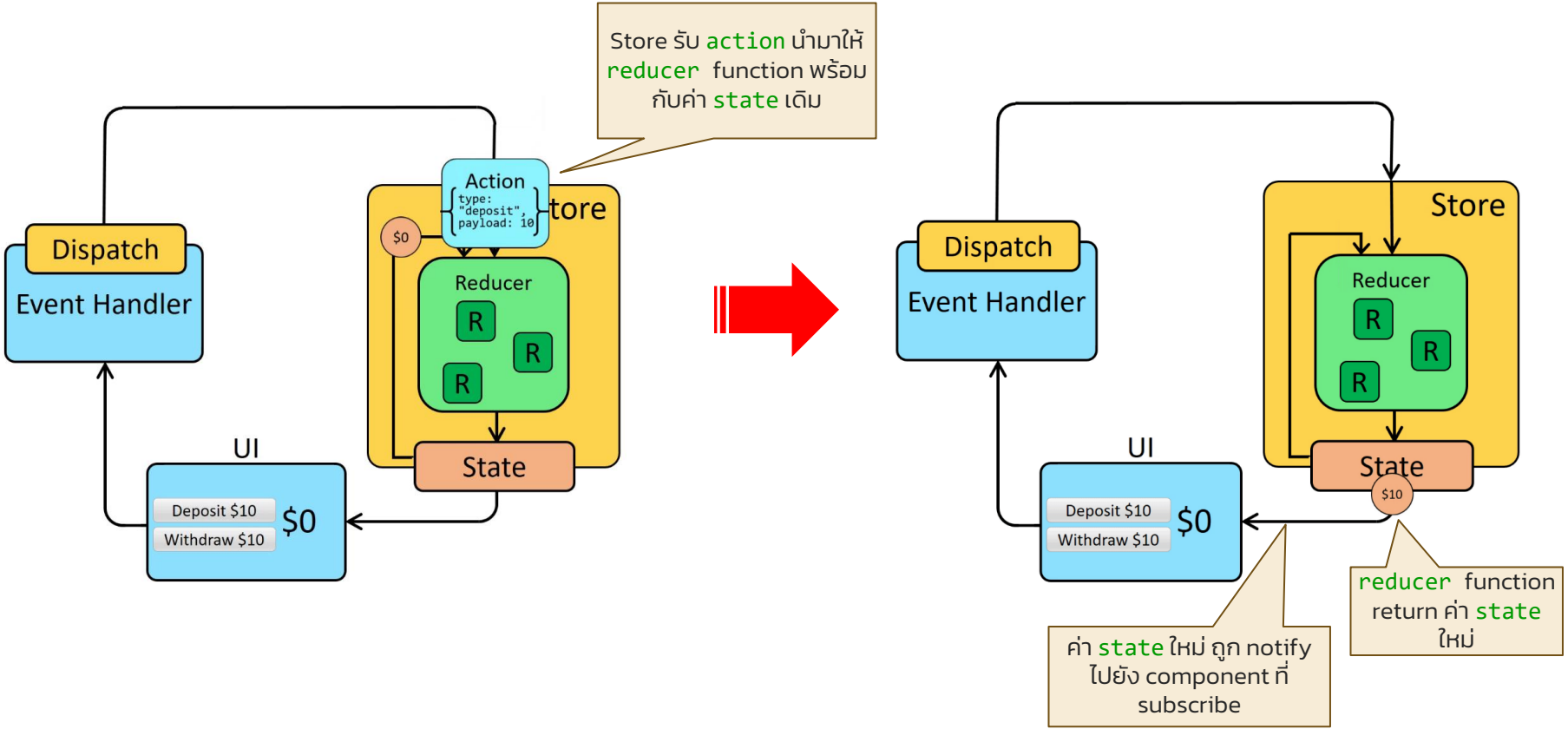
- **ช่วง Initial setup:**
  - Redux store ถูกสร้างด้วย root reducer function
  - Store เรียก root reducer 1 ครั้ง แล้ว save ค่า return value เป็นค่าเริ่มต้นของ state
  - ในการ render UI ครั้งแรกนั้น UI component ใช้ค่าปัจจุบันของ state ใน store เพื่อประกอบการ render
  - UI Component ทำการ subscribe รับข้อมูล store update เพื่อรับทราบถึงการเปลี่ยนแปลงของ state

- **ช่วงการ Updates:**
  - มีเหตุการณ์เกิดขึ้นใน app เช่น user click ปุ่ม ทำให้ app code ทำการ dispatch action ไปยัง store
  - Store run reducer function อีกครั้ง โดยใช้ค่า state เดิม และ action ปัจจุบัน จากนั้น save ค่า return value เป็นค่า state ใหม่
  - Store ทำการ notify ทุกส่วนของ UI ที่ subscribe ต่อ store update
  - แต่ละ UI component ที่ต้องการ data จาก store ตรวจสอบว่าส่วนของ state ที่ใช้ มีการเปลี่ยนแปลงหรือไม่ ถ้ามีการเปลี่ยนแปลง จากทำการ re-render ด้วย data ใหม่ เพื่อตอบสนองต่อการ update

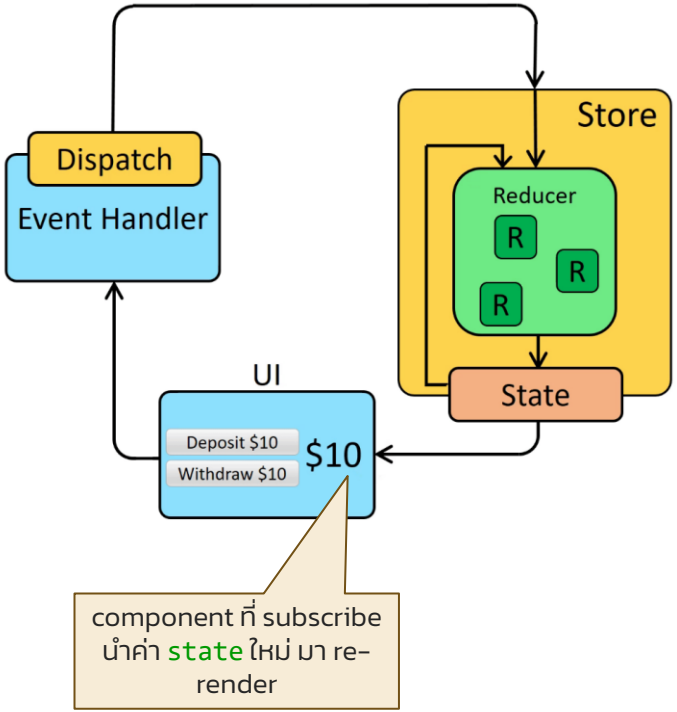
Redux Concepts – *How it works?*



Redux Concepts – *How it works?*



Redux Concepts – *How it works?*



Redux Libraries and Tools



- Redux Libraries and Tools
  - Redux เป็น state management JavaScript library แบบ standalone
    - ต้องการ UI binding library สำหรับจัดการการเชื่อมต่อและจัดการ interaction logic ระหว่าง Redux Library กับ front-end framework
  - React-Redux เป็น official Redux UI binding library สำหรับ React application โดย Redux team
    - เอื้อให้ React component สามารถ interact กับ Redux store โดยการอ่าน state และการ dispatch action ไป update store
  - Redux Toolkit เป็น Tools ที่ให้รูปแบบใหม่ในการพัฒนา Redux logic



    - มาพร้อม package และ function จำเป็นสำหรับการสร้าง Redux app
    - การใช้ Redux Toolkit เป็น best practice สำหรับการใช้งาน Redux ทำให้ทำงานกับ Redux ง่ายขึ้น และหลีกเลี่ยงข้อผิดพลาด
  - Redux DevTools Extension เป็น Tools ส่วนขยายสำหรับ Redux
    - แสดง history การเปลี่ยนแปลง state ใน Redux store
    - ช่วยในการ debug application

Redux Toolkit



- **Redux Toolkit** มาพร้อม package และ function ที่จำเป็นสำหรับการสร้าง Redux App
  - วางรูปแบบมาตรฐานสำหรับการเขียน Redux Logic
  - ใช้งานโดยการติดตั้ง **ReduxJS Toolkit** และ **React Redux**

```
npm i @reduxjs/toolkit react-redux
```

- **Store Setup**
  - Redux Toolkit มี `configureStore` API สำหรับ setup Redux Store โดย wrap การใช้งาน `createStore` API ดั้งเดิมของ Redux เพื่อ setup Store โดยอัตโนมัติ

```
import { configureStore } from '@reduxjs/toolkit'
…
const store = configureStore({
  reducer: {
    // Top-level state fields with Reducer State Slices that handle them
    …
  }
})
export default store
```

รายการของ state คู่กับ state slices สำหรับจัดการ state นั้น ๆ

Redux Toolkit



- State Slices
  - Redux state slices เป็น component ที่ครอบ logic สำหรับการบริหารจัดการ state ของ data item ภายใน redux store ทำให้ง่ายต่อการจัดการ Redux reducer logic และ actions
  - Redux Toolkit มาพร้อม `createSlice` API ที่ช่วยให้
    - Developer เขียน reducer function แยกเป็น case ตามที่ต้องการจัดการ ไว้ภายใน Object
    - ง่ายต่อการเขียน immutable update logic
    - Generate action creator ให้อัตโนมัติ โดยพิจารณาจาก reducer function ที่กำหนดไว้

  - `createSlice` รับ 3 object เป็น main option argument
    - `name`: ค่า string ที่จะเป็น prefix สำหรับสร้าง action type ต่าง ๆ
    - `initialState`: ค่า initial state สำหรับ reducer
    - `reducers`: เป็น object บรรจุ reducer function ต่าง ๆ แยกตาม case เพื่อจัดการ action ตาม case

Redux Toolkit



- **State Slices**
  - ตัวอย่าง การใช้งาน `createSlice`

```
import { createSlice } from '@reduxjs/toolkit'

const initialState = {
  entities: [],
  status: null
}
const todosSlice = createSlice({
  name: 'todos',
  initialState,
  reducers: {
    todoAdded(state, action) {
          state.entities.push(action.payload)
    },
    todoToggled(state, action) {
      const todo=state.entities.find(todo => todo.id===action.payload)
      todo.completed = !todo.completed
    }
  }
})

export const { todoAdded, todoToggled} = todosSlice.actions
export default todosSlice.reducer
```

ค่า initial state สำหรับ reducer

ค่า string ที่จะเป็น prefix สำหรับสร้าง action type ต่าง ๆ

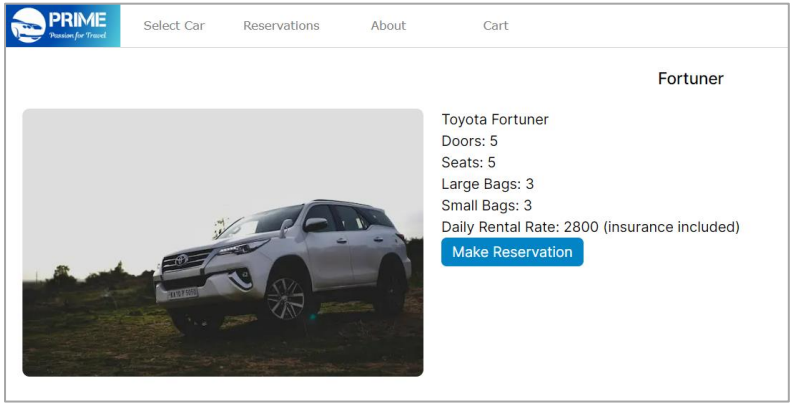
Reducer Function Case `todoAdded`

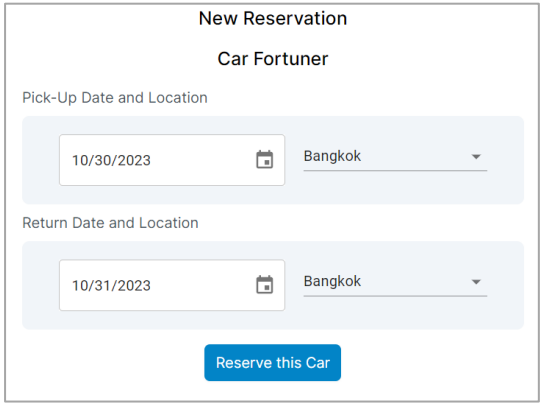
Reducer Function Case `todoToggled`

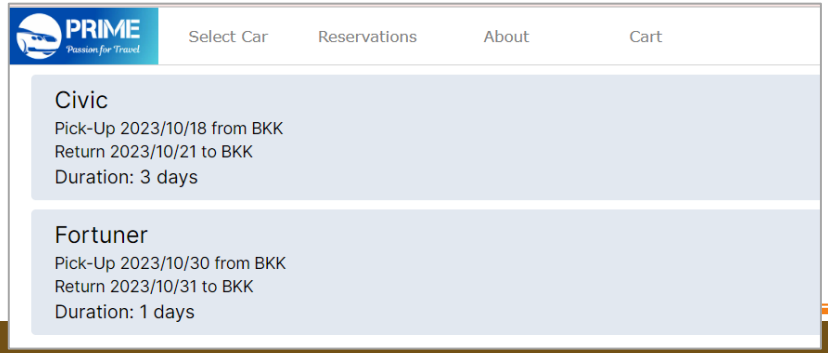
export action creator ที่ generate ขึ้นมา เข้าถึงจาก component อื่นได้ ผ่าน
`slice.actions.todoAdded`
`slice.actions.todoToggled`

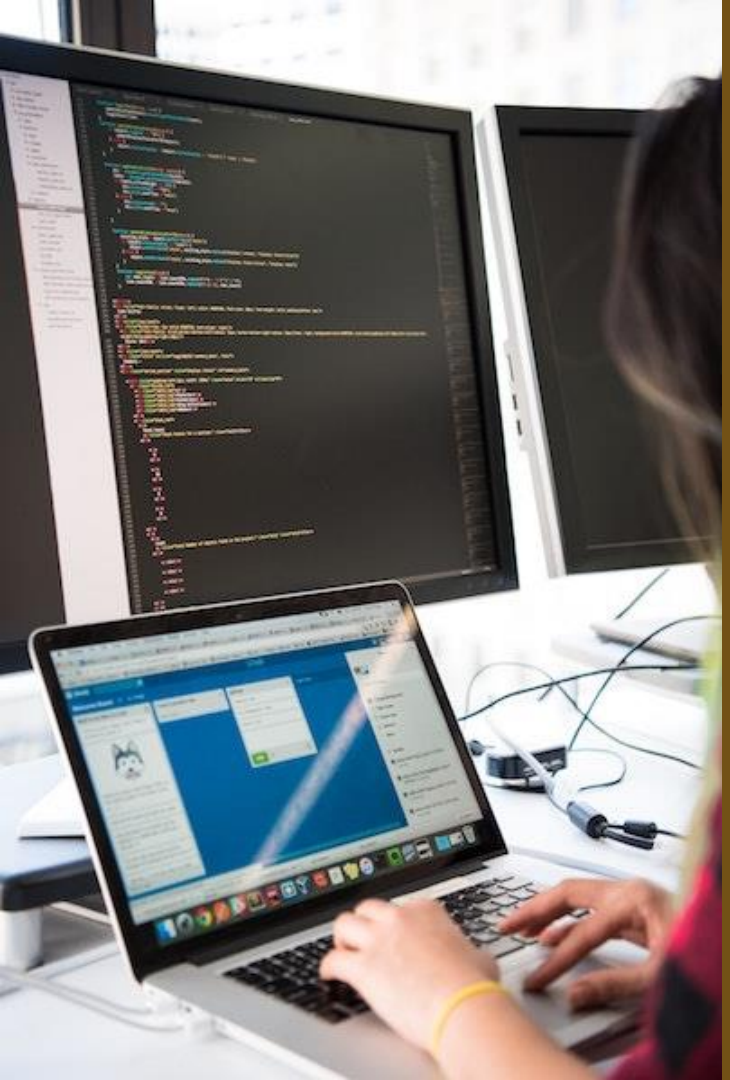
Workshop 11.1 – 11.2

- เราจะใช้ Redux Toolkit สำหรับทำ State Management เพื่อสร้าง Shopping Cart สำหรับ Order การจองรถเช่า









Workshop 11.1: Using Redux Toolkit for State Management

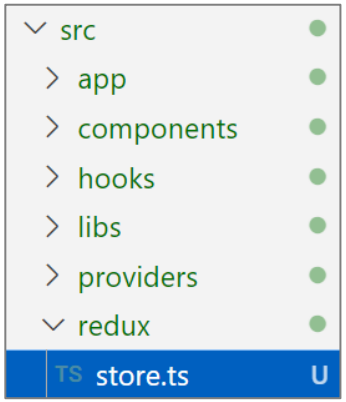
Using Redux Toolkit



- Stop Server และ ติดตั้ง Redux Toolkit ผ่าน console ด้วยคำสั่ง

```
npm i @reduxjs/toolkit react-redux
```

- สร้าง directory สำหรับเก็บไฟล์ที่เกี่ยวข้องกับการใช้งาน Redux
  - สร้างไดเร็กทอรี่ `src/redux/` และสร้างไฟล์ `store.ts` สำหรับเก็บ Redux Store




```ts
import { configureStore } from "@reduxjs/toolkit";

export const store = configureStore({
    reducer:{

    }
})

export type RootState = ReturnType<typeof store.getState>
export type AppDispatch = typeof store.dispatch
```

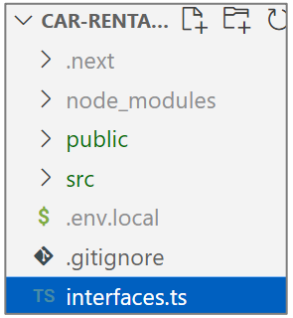
สำหรับระบุ `state slice` ต่าง ๆ

TypeScript ต้องการ การระบุ type จึง export
- type ของ state ที่ return จาก store นี้ โดยนิยามเป็น type ชื่อ `RootState`
- type ของ store dispatch โดยนิยามเป็น type ชื่อ `AppDispatch`

เพื่อให้ component อื่นรู้จัก type เหล่านี้ และนำไปใช้ในการเข้าถึง state ได้

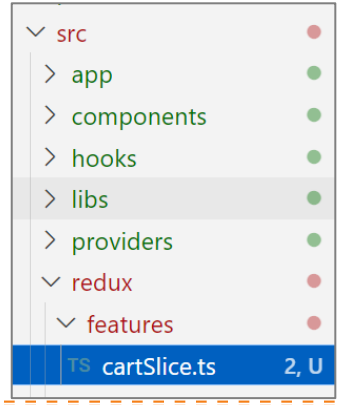
Using Redux Toolkit

- ที่ Root Directory ของ project นิยาม Interface สำหรับระบุโครงสร้างของ reservation item
  - สร้างไฟล์ `interfaces.ts`




```ts
export interface ReservationItem {
    carId: string
    carModel: string
    numOfDays: number
    pickupDate: string
    pickupLocation: string
    returnDate: string
    returnLocation: string
}
```

- ที่ไดเร็กทอรี่ `src/redux/`
  - สร้างไดเร็กทอรี่ `features` และไฟล์ `cartSlice.ts` สำหรับเก็บ State Slice




```ts
import { createSlice } from "@reduxjs/toolkit"

export const cartSlice = createSlice({
    name: "cart",
})
```

Using Redux Toolkit



- ไฟล์ `cartSlice.ts`

```ts
import { createSlice } from "@reduxjs/toolkit"
import { ReservationItem } from "../../../interfaces"

type CartState = {
    carItems: ReservationItem[]
}

const initialState:CartState = { carItems: []}

export const cartSlice = createSlice({
    name: "cart",
    initialState,
    reducers: {

    }
})
```

นิยาม type ชื่อ `CartState` เป็น Object ที่มี attribute `cartItems` เก็บ array ของ `ReservationItem`

สร้าง `initialState` เป็น Object ชนิด `CartState` ที่ `cartItems` เป็น empty array

สร้าง `slice` ที่มี `initialState` เป็น state เริ่มต้น

Using Redux To

- สร้าง Reducer Function ให้กับ cartSlice

```
    initialState,
    reducers: {
        addReservation: (state, action:PayloadAction<ReservationItem>)=>{
            state.carItems.push(action.payload)

        },
        removeReservation: (state, action:PayloadAction<ReservationItem>)=>{
            const remainItems = state.carItems.filter(obj => {
                return ((obj.carModel !== action.payload.carModel)
                ||(obj.pickupDate !== action.payload.pickupDate)
                ||(obj.returnDate !== action.payload.returnDate));
            })
            state.carItems = remainItems
        }
    }
})
export const { addReservation, removeReservation } = cartSlice.actions
export default cartSlice.reducer
```

- เพิ่ม cartSlice เข้าไปใน store.ts

```
import cartSlice from "./features/cartSlice";
export const store = configureStore({
    reducer:{
        cartSlice
    }
})
```

Access State with Redux Toolkit



การใช้งาน Redux State ผ่าน `Provider`

- react-redux `Provider` เป็นส่วนที่ทำให้ Nested Component ต่าง ๆ เข้าถึง Redux store ได้
  - ในการเข้าถึง Store นั้น component ต้องถูกครอบด้วย `<Provider> … </Provider>`
    - นิยมนำ `<Provider> … </Provider>` ไป wrap ทั้ง application เพื่อให้สามารถเข้าถึง Redux Store ได้จากทั้ง component tree
  - ในการใช้งาน `Provider` ส่ง `store` ให้กับ prop `store` ของ `Provider`

```
<Provider store={store}> … </Provider>
```

Access State with Redux Toolkit



- การเข้าถึง Redux State
  - ใน Next.js จะเข้าถึง Redux state จาก Client Component โดย React Redux มาพร้อม custom hook API ที่เอื้อให้ React component สามารถ subscribe กับ Redux store และ dispatch action ได้
  - ในการเข้าถึง Store นั้น component ต้องถูกครอบด้วย `<Provider> … </Provider>`
    - นิยมนำ `<Provider> … </Provider>` ไป wrap ทั้ง application เพื่อให้สามารถเข้าถึง Redux Store ได้จากทั้ง component tree
  - `useSelector()` **Hook** ใช้สำหรับ extract data จาก Redux store state เพื่อใช้งานใน component ผ่าน `selector` function

```
value = useSelector( selectorFunction )
```

    - *selectorFunction* เป็น Selector Function ที่รับ Redux store state ทั้งชิ้นเป็น argument สามารถ return ค่าในรูปแบบใดออกมาก็ได้ แต่นิยม return ค่า value ภายใน state ออกมา
    - *value* ค่าที่ return มาจาก *selectorFunction*

```
const counter = useSelector((state) => state.counter)
```

Access State with Redux Toolkit



- `useSelector()` Hook
  - Selector จะถูก run เมื่อ functional component ได้รับการ render
    - ยกเว้นเมื่อ reference ของ selector ไม่มีการเปลี่ยนแปลงนับจากการ render ก่อนหน้า ในกรณีนี้ค่าที่ถูก cached ไว้จะเป็นค่าที่ return จาก hook โดยไม่มีการ re-running selector
  - `useSelector()` จะ subscribe ไปที่ Redux store ด้วย และจะ run selector เมื่อ action ถูก dispatch
    - เมื่อ action ถูก dispatch จะทำให้ `useSelector()` ทำการเปรียบเทียบค่า value ปัจจุบันของ reference ของ selector เทียบกับค่า value ก่อนหน้า ถ้ามีความต่างกัน จะทำให้ component เกิดการ re-render แต่ถ้าเหมือนกัน component จะไม่ re-render
    - ในการเปรียบเทียบ โดย default จะใช้ strict === reference equality check

Dispatch Action with Redux Toolkit



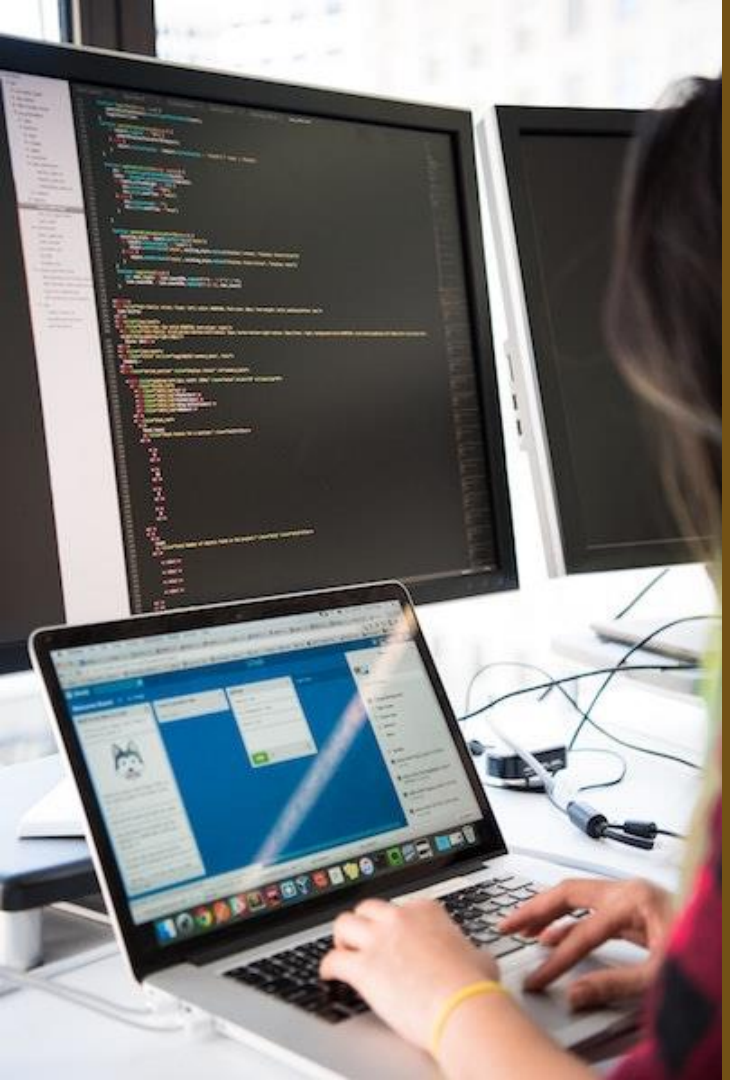
- `useDispatch()` Hook คืนค่า reference ไปยัง `dispatch` function จาก Redux store ใช้สำหรับการ dispatch action เมื่อต้องการ
  - ตัวอย่าง

```
import React from 'react'
import { useDispatch } from 'react-redux'

export const CounterComponent = ({ value }) => {
  const dispatch = useDispatch()

  return (
    <div>
      <span>{value}</span>
      <button onClick={() => dispatch({ type: 'increment-counter' })}>
        Increment counter
      </button>
    </div>
  )
}
```

action ที่ dispatch



Workshop 11.2: Accessing Redux State

Using Provider

- ใน src/redux/
  - สร้างไฟล์ ReduxProvider.tsx เป็น client component สำหรับเก็บ Redux Custom Provider

```tsx
"use client"
import { store } from '@/redux/store'
import {Provider as ReactReduxProvider} from 'react-redux'

export default function ReduxProvider({children}: {children:React.ReactNode}) {
    return (
        <ReactReduxProvider store={store}>
        {children}
        </ReactReduxProvider>
    )
}
```

- ใน app/layout.tsx
  - นำ ReduxProvider มาครอบ component ของ application

```tsx
import TopMenu from '@/components/TopMenu'
import { getServerSession } from 'next-auth'
import { authOptions } from './api/auth/[...nextauth]/route'
import NextAuthProvider from '@/providers/NextAuthProvider'
import ReduxProvider from '@/redux/ReduxProvider'
```

```tsx
const session = await getServerSession(authOptions)
return (
  <html lang="en" className={`${geistSans.variable} ${g
    <body>
      <ReduxProvider>
      <NextAuthProvider session={session}>
        <TopMenu/>
        {children}
      </NextAuthProvider>
      </ReduxProvider>
    </body>
  </html>
);
```

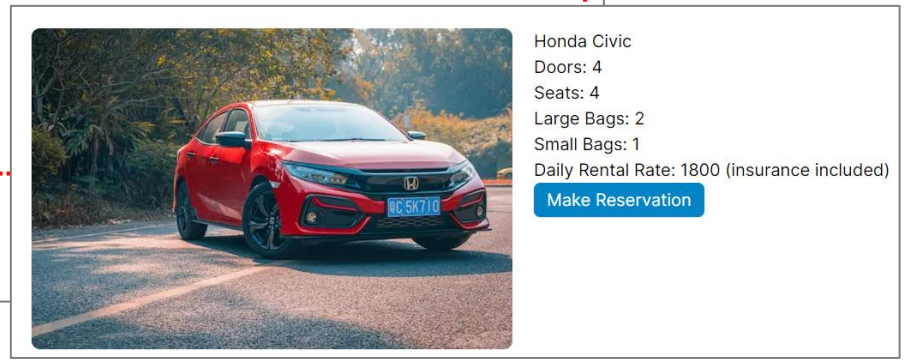
Dispatching Action

- เพิ่มการจองรถใน route http://localhost:3000/car/{carId}
  - ใน app\(carinfo)\car\[cid]\page.tsx
  - เพิ่มปุ่มที่เป็น Link ไปยังหน้า http://localhost:3000/reservations โดยส่ง car id และ model เป็น Query Parameter ไปให้

```
<div>Daily Rental Rate: {carDetail.data.dayRate} (insurance included)</div>

<Link href={`/reservations?id=${cid}&model=${carDetail.data.model}`}>
<button className="block rounded-md bg-sky-600 hover:bg-indigo-600 px-3 py-1
text-white shadow-sm">
    Make Reservation
</button>
</Link>

</div>
</div>
```



Dispatching Action

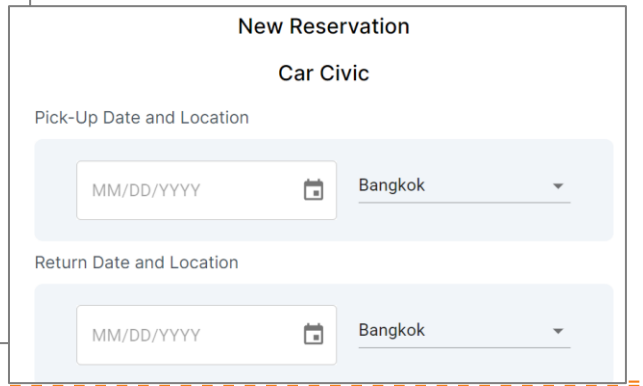
- ปรับ Page `app\reservations\page.tsx` ของ route http://localhost:3000/reservations
  - เป็น Client Component
  - รองรับการเข้าถึง Query Parameter โดยเป็น page ที่ dynamic ตามข้อมูล car ที่ user เลือกเพื่อจองรถ

```tsx
"use client"
import LocationDateReserve from "@/components/LocationDateReserve";
import { useSearchParams } from "next/navigation";

export default function Reservations () {

    const urlParams = useSearchParams()
    const cid = urlParams.get('id')
    const model = urlParams.get('model')


    return (
        <main className="w-[100%] flex flex-col items-center space-y-4">
            <div className="text-xl font-medium">New Reservation</div>
            <div className="text-xl font-medium">Car {model}</div>
```



Dispatching Action

- ใน components\LocationDateReserve.tsx

รับ onDateChange และ onLocationChange handler ที่ parent component ส่งมาให้ เพื่อ handle เมื่อค่า date เปลี่ยน และเมื่อค่า location เปลี่ยน

```
import { Dayjs } from "dayjs"

export default function LocationDateReserve({onDateChange, onLocationChange}
    :{onDateChange:Function, onLocationChange:Function}) {

    const [reserveDate, setReserveDate] = useState<Dayjs|null>(null)
    const [location, setLocation] = useState('BKK')

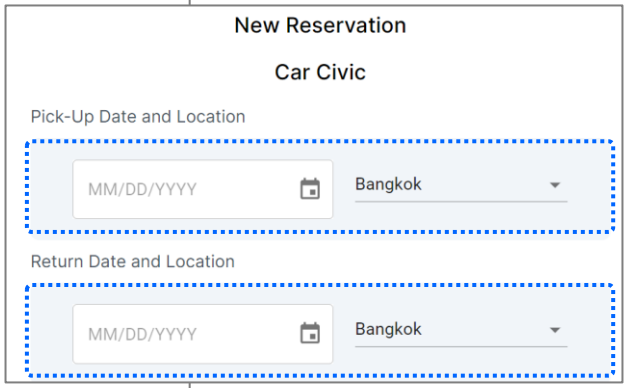
    return (
        <div className="bg-slate-100 rounded-lg space-x-5 space-y-2
        w-fit px-10 py-5 flex flex-row justify-center">

            <LocalizationProvider dateAdapter={AdapterDayjs}>
                <DatePicker className="bg-white"
                value={reserveDate}
                onChange={(value)=>{setReserveDate(value); onDateChange(value)}}
                />
            </LocalizationProvider>

            <Select variant="standard"
            name="location" id="location" value={location}
            onChange={ (e)=> { setLocation(e.target.value); onLocationChange(e.target.value);} }
```

เรียก onDateChange handler ที่รับเข้ามา เมื่อค่า Date เปลี่ยน

เรียก onLocationChange handler ที่รับเข้ามา เมื่อค่า Location เปลี่ยน



Dispatching Action



- ใน app\reservations\page.tsx
  - สร้าง state มาเก็บค่า pickup date, pickup location, return date, return location
  - ส่งการ set state เป็น handler onDateChange และ onLocationChange ของ LocationDateReserve

```tsx
import { useState } from "react";
import { Dayjs } from "dayjs";

export default function Reservations () {

    const urlParams = useSearchParams()
    const cid = urlParams.get('id')
    const model = urlParams.get('model')

    const [pickupDate, setPickupDate] = useState<Dayjs|null>(null)
    const [pickupLocation, setPickupLocation] = useState<string>("BKK")
    const [returnDate, setReturnDate] = useState<Dayjs|null>(null)
    const [returnLocation, setReturnLocation] = useState<string>("BKK")

    return (
        <main className="w-[100%] flex flex-col items-center space-y-4">
            <div className="text-xl font-medium">New Reservation</div>
            <div className="text-xl font-medium">Car {model}</div>

            <div className="w-fit space-y-2">
                <div className="text-md text-left text-gray-600">Pick-Up Date and Location</div>
                <LocationDateReserve onDateChange={(value:Dayjs)=>{setPickupDate(value)}}
                onLocationChange={(value:string)=>setPickupLocation(value)}/>
                <div className="text-md text-left text-gray-600">Return Date and Location</div>
                <LocationDateReserve onDateChange={(value:Dayjs)=>setReturnDate(value)}
                onLocationChange={(value:string)=>setReturnLocation(value)}/>
            </div>
```

Dispatching Action



- ใน app\reservations\page.tsx
  - เมื่อกด reserve แล้วจะ dispatch ข้อมูล reservation item ไปให้ Cart
  - ใช้งาน useDispatch สำหรับ type AppDispatch

```
import { Dayjs } from "dayjs";
import { useDispatch } from "react-redux";
import { AppDispatch } from "@/redux/store";

export default function Reservations () {

    const urlParams = useSearchParams()
    const cid = urlParams.get('id')
    const model = urlParams.get('model')

    const dispatch = useDispatch<AppDispatch>()

    const [pickupDate, setPickupDate] = useState<Dayjs|null>(null)
```

Dispatching Action

- ใน app\reservations\page.tsx
  - สร้างฟังก์ชั่น MakeReservation ไว้สำหรับ สร้าง ReservationItem และ dispatch ไปให้ Cart
  - เปลี่ยนปุ่ม Check Car Availability เป็น Reserve this Car โดยเมื่อคลิกเรียก MakeReservation

```
    onLocationChange={(value:string)=>setReturnLo
</div>

<button className="block rounded-md bg-sky-600 ho
text-white shadow-sm" onClick={makeReservation}>
    Reserve this Car
</button>
ain>
```

```
import dayjs, { Dayjs } from "dayjs";
import { useDispatch } from "react-redux";
import { AppDispatch } from "@/redux/store";
import { addReservation } from "@/redux/features/cartSlice";
import { ReservationItem } from "../../../interfaces";

export default function Reservations () {

    const urlParams = useSearchParams()
    const cid = urlParams.get('id')
    const model = urlParams.get('model')
    const dispatch = useDispatch<AppDispatch>()

    const makeReservation = () => {
        if(cid && model && pickupDate && returnDate) {
            const item:ReservationItem = {
                carId: cid,
                carModel: model,
                numOfDays: returnDate.diff(pickupDate,'day'),
                pickupDate: dayjs(pickupDate).format("YYYY/MM/DD"),
                pickupLocation: pickupLocation,
                returnDate: dayjs(returnDate).format("YYYY/MM/DD"),
                returnLocation: returnLocation
            }
            dispatch(addReservation(item))
        }
    }
```

Accessing Redux State



- ใน `store.ts`
  - สร้าง Selector สำหรับให้ component อื่นใช้งาน `useSelector` ที่มี type สำหรับเข้าถึง `RootState` ของ Store

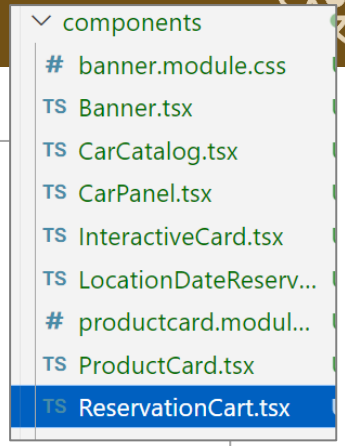
```
import { configureStore } from "@reduxjs/toolkit";
import cartSlice from "./features/cartSlice";
import { useSelector, TypedUseSelectorHook } from "react-redux";
export const store = configureStore({
    reducer:{
        cartSlice
    }
})

export type RootState = ReturnType<typeof store.getState>
export type AppDispatch = typeof store.dispatch
export const useAppSelector: TypedUseSelectorHook<RootState> = useSelector
```

Accessing Redux State

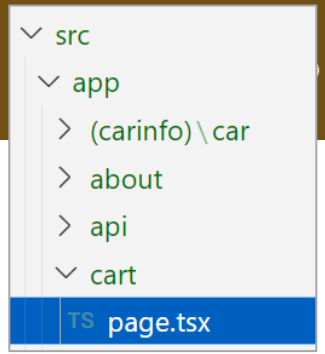


- สร้าง components\ReservationCart.tsx เพื่อแสดงข้อมูลใน Cart




```
"use client"
import { useAppSelector } from "@/redux/store"
export default function ReservationCart () {
    const carItems = useAppSelector((state)=> state.cartSlice.carItems)
    return (
        <>
        {
            carItems.map((reservationItem)=> (
                <div className="bg-slate-200 rounded px-5 mx-5 py-2 my-2" key={reservationItem.carId}>
                <div className="text-xl">{reservationItem.carModel}</div>
                <div className="text-sm">Pick-Up {reservationItem.pickupDate}
                from {reservationItem.pickupLocation}</div>
                <div className="text-sm">Return {reservationItem.returnDate}
                sto {reservationItem.returnLocation}</div>
                <div className="text-md">Duration: {reservationItem.numOfDays} days</div>
                </div>
            ))
        }
        </>
    )
}
```

Accessing Redux State



- สร้าง Page `app\cart\page.tsx` ของ route http://localhost:3000/cart
  - แสดง `ReservationCart`

```tsx
"use client"
import ReservationCart from "@/components/ReservationCart"
export default function CartPage () {
    return (
        <main>
            <ReservationCart></ReservationCart>
        </main>
    )
}
```

Accessing Redux State

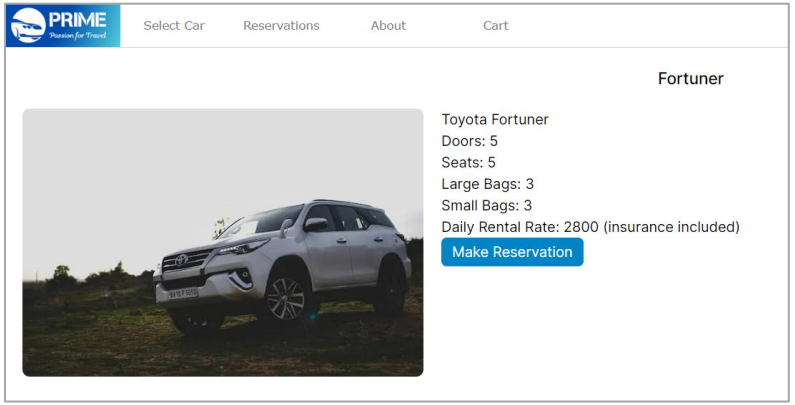
- ที่ components\TopMenu.tsx สร้างเมนูเพื่อเข้าถึง route /cart และ route /car

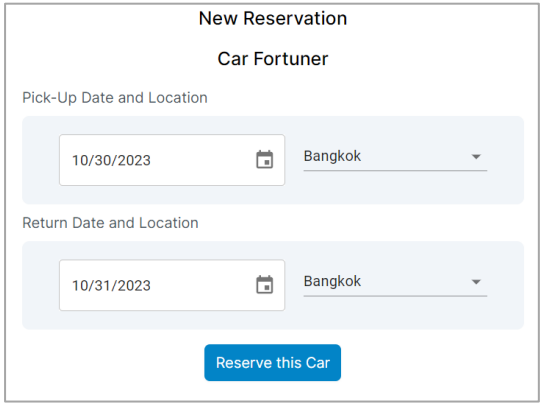
```
<div className={styles.menucontainer}>
    <Image src={'/img/logo.png'} className={styles.logoimg}
    alt='logo' width={0} height={0} sizes="100vh"/>
    <TopMenuItem title='Select Car' pageRef='/car'/>
    <TopMenuItem title='Reservations' pageRef='/reservations'/>
    <TopMenuItem title='About' pageRef='/about'/>
    <TopMenuItem title='Cart' pageRef='/cart'/>
    {
        session? <Link href="/api/auth/signout">
            <div className='flex items-center absolute right-0 h-full px-2 text-cyan-600 text-sm'>
            Sign-Out of {session.user?.name} </div></Link>
            : <Link href="/api/auth/signin">
            <div className='flex items-center absolute right-0 h-full px-2 text-cyan-600 text-sm'>
            Sign-In</div></Link>
    }
</div>
```

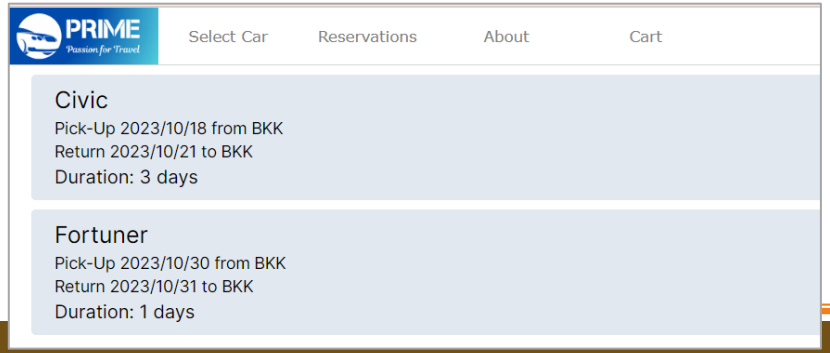


Adding to Cart

- ทดลอง Make Reservation เพื่อ Add Reservation Item มายัง Cart และแสดง Cart







Removing from Cart

- ใน components\ReservationCart.tsx เพิ่มการ dispatch action removeReservation เมื่อกดปุ่ม

```
"use client"
import { removeReservation } from "@/redux/features/cartSlice"
import { AppDispatch, useAppSelector } from "@/redux/store"
import { useDispatch } from "react-redux"
export default function ReservationCart () {
    const carItems = useAppSelector((state)=> state.cartSlice.carItems)
    const dispatch = useDispatch<AppDispatch>()
    return (
        <>
        {
            carItems.map((reservationItem)=> (
                <div className="bg-slate-200 rounded px-5 mx-5 py-2 my-2" key={reservationItem.carId}>
                <div className="text-xl">{reservationItem.carModel}</div>
                <div className="text-sm">Pick-Up {reservationItem.pickupDate}
                from {reservationItem.pickupLocation}</div>
                <div className="text-sm">Return {reservationItem.returnDate}
                to {reservationItem.returnLocation}</div>
                <div className="text-md">Duration: {reservationItem.numOfDays} days</div>
                <button className="block rounded-md bg-sky-600 hover:bg-indigo-600 px-3 py-1
                text-white shadow-sm" onClick={()=>dispatch(removeReservation(reservationItem))}>
                    Remove from Cart
                </button>
                </div>
            ))
        }
```

Removing from Cart

- ทดลอง Make Reservation เพื่อ Add Reservation Item มายัง Cart และแสดง Cart

